

In-Memory Sorting and Searching with Akers Array

Kasturi Dikpati
Information Technology
University Institute of Technology
Burdhawan, India
kasturi.dikpati@gmail.com

Barnik Roy
Computer Science and Technology
IIEST, Shibpur
Howrah, India
thunderburning987@gmail.com

Pantho Propan Debnath
Computer Science and Technology
IIEST, Shibpur
Howrah, India
panthonath12179@gmail.com

Biplab K. Sikdar
Computer Science and Technology
IIEST, Shibpur
Howrah, India
biplab@cs.iests.ac.in

Abstract—In the realm of efficient data management, sorting and searching algorithms play a crucial role in optimizing performance. This work presents a comprehensive study of in-memory sorting and searching using the Akers logic array, a two-dimensional structure capable of parallel computation through configurable logic cells. We extend the capabilities of the Akers array beyond binary sorting to support generalized (non-binary) sorting, and further implement both linear and binary search operations within the same framework. A conceptual memory architecture is also proposed, partitioning the Akers array into Stasis and Flux regions to differentiate between storage and computation zones. Our approach reveals significant improvements in execution time and resource efficiency when implemented in hardware, offering a scalable and reconfigurable solution for high-performance, data-centric applications. These insights pave the way for future innovations in in-memory computing and hardware-level algorithm optimization.

Index Terms—Akers array, in-memory-computation, sorting, cellular logic, combinatorial logic, parallel processing.

I. INTRODUCTION

In recent years, computing architectures have undergone significant transformation, moving away from the conventional von Neumann model toward innovative designs that integrate processing and memory functionalities and effectively realized in-memory computation that conventionally done by the CPU. The Akers logic array, introduced by Sheldon Akers in 1972 [1], which facilitates the execution of Boolean functions through a network of primitive logic cells. Each cell within this grid features three inputs and two outputs, facilitating the dynamic introduction of external input variables.

This work presents a two-dimensional array of identical logic cells designed to meet these evolving requirements while retaining the beneficial characteristics of Akers arrays. Through careful configuration of input variables across the array, a systematic method for synthesizing combinatorial switching functions is developed that leads to realization of in-memory sorting.

To showcase the practicality of the Akers logic array in data-intensive operations, the following sections delve into

its core components and demonstrate its adaptability for in-memory computation. The paper begins by laying out the architecture of the Akers array, describing the structure and behavior of its logic cells, and how these cells interconnect to enable parallel execution of Boolean functions. With this foundation, we explore its application in sorting binary inputs using a triangular configuration and control logic, establishing the array's baseline utility in basic in-memory operations.

Building upon binary sorting, the paper then transitions to more advanced use cases—namely, generalized sorting with non-binary symbols and implementing comparative operations using binary-coded data. These sections reveal how the Akers array supports the median function for multi-valued logic and facilitates fundamental sorting steps like min-max extraction. The analysis continues with a detailed treatment of search operations—both linear and binary—realized within the same logic framework, culminating in the conceptualization of a hybrid memory model with partitioned zones for storage (Stasis) and computation (Flux). Together, these sections underscore the versatility of Akers arrays in redefining memory and computation boundaries in modern hardware design.

II. AKERS ARRAY

Akers array has been introduced by S. B. Akers in 1972. Such an Akers array is composed of identical logic cells interconnected in a rectangular grid, as shown in Figure 1. Each cell has three inputs and two outputs. Two of the inputs are fixed connections — immediately from up (X) and left (Y). The two outputs (f) are permanently connected to the two cells — immediately to the right and down. The third input to the cell is an external input on which an arbitrary input variable (Z) may be introduced. This input is under the designer's control. By defining logical operation of the array cells in proper ways, several logic functions can be realized. However, it is to be expected that certain input combinations to the cells never occur.

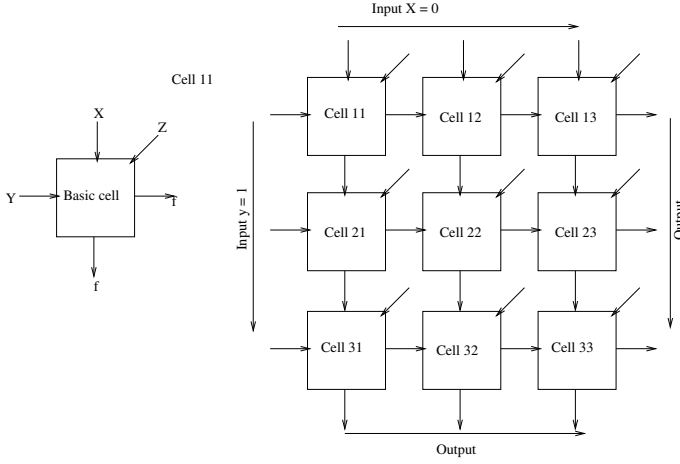


Fig. 1: Akers array

A binary Akers array cell implements only one out of the following four possible Boolean functions, $f(x, y, z) : \{0, 1\}^3 \rightarrow \{0, 1\}$, as defined in [1]:

$$\begin{aligned} f &= y(x + z) \\ f &= z'x + zy \\ f &= xy + yz + zx \\ f &= x + yz \end{aligned}$$

III. BINARY SORTING WITH AKERS ARRAY

Binary-sorting (bit sorting) with Akers array has been introduced in [2]. A binary bit sequence of n elements is $[x_1, \dots, x_n]$. A triangular configuration above the minor diagonal of a square matrix of order n , as shown in Figure 2), is introduced to realize the sorting procedure. Here, the cells are denoted as (i, j) under the condition $i + j \leq n + 1$. The control input for a cell (i, j) corresponds to x_{i+j-1} . The outputs from the last cell of each row form the sorted sequence of the input sequence.

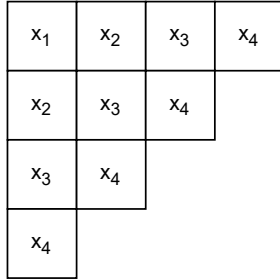


Fig. 2: Akers array for bit-wise sorting [2]

IV. GENERALIZED SORTING

A. Sorting with non-binary Akers Array

Previous scientific research [3] explores that the properties of binary Akers arrays can be theoretically extended to non-binary Akers arrays with some additional features. To sort n numbers, $\frac{n \times (n+1)}{2}$ basic cells are required. Figure 3 illustrates such a scenario with 4 non-binary symbols which requires $\frac{4 \times (4+1)}{2} = 10$ cells. The non-binary alphabets can be considered as $\mathcal{A} = \{0, 1, 2, \dots, q-1\}$ where $q \in \mathbf{Z}^+$.

The basic cell output can be defined as a function, $f(x, y, z) : \mathcal{A}^3 \rightarrow \mathcal{A}$, given by:

$$f = \text{median}(x, y, z)$$

All cells of the leftmost column take $q-1$ as the y (left) input. Conversely, all cells of the topmost row receive 0 as the x (top) input. The numbers to be sorted are provided as the variable inputs z . It can be observed from Figure 3 that $[z_0, z_1, z_2, z_3]$ is the sequence to be sorted and $[f_3, f_6, f_8, f_9]$ is the sorted sequence.

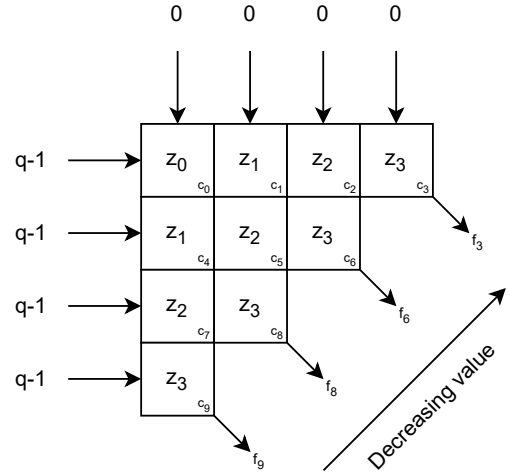


Fig. 3: Non-binary Akers array

The following two examples (in octal number system) illustrate the sorting.

Example 1: Let $z_0 = 6$, $z_1 = 3$, $z_2 = 4$, and $z_3 = 1$. The sorted output yields:

$$f_3 = 1, \quad f_6 = 3, \quad f_8 = 4, \quad \text{and} \quad f_9 = 6$$

The f for individual cells are shown in Table I.

Example 2: Let $z_0 = 1$, $z_1 = 4$, $z_2 = 3$, and $z_3 = 6$. The sorted output:

$$f_3 = 1, \quad f_6 = 3, \quad f_8 = 4, \quad \text{and} \quad f_9 = 6$$

The f for individual cells are shown in Table II.

TABLE I: The f for Example 1

#cell	output
0	6
1	3
2	3
3	1
4	6
5	4
6	3
7	6
8	4
9	6

TABLE II: The f for Example 2

#cell	output
0	1
1	1
2	1
3	1
4	4
5	3
6	3
7	4
8	4
9	6

B. Using binary Akers array

The generalized sorting described in Section IV-A demands the computation of $median(x, y, z)$. This section realizes generalized sorting with binary Akers array. Here, the elements of the input list are sorted in n -bit binary format.

The 1st step of sorting a list of numbers (elements) is a comparison between two numbers. Let assume two n -bit binary numbers:

- $A = A_{n-1}A_{n-2} \dots A_1A_0$
- $B = B_{n-1}B_{n-2} \dots B_1B_0$

The numbers to be sorted are linearly stored in memory.

The i^{th} bits of A and B - that is, A_i and B_i are compared. The truth table for $A_i > B_i$ is shown in Table III.

TABLE III: Truth table for $A_i > B_i$

A_i	B_i	$A_i > B_i$
0	0	0
0	1	0
1	0	1
1	1	0

The SOPs (sum of products) A_iB_i and POSs (product of sums) of A_i and B_i are derived using Karnaugh maps as shown in Figure 4.

In Table IV the SOPs (1s) are assigned as “row headers”, and the POSs (0s) are assigned as the “column headers”. The table entries are:

$$entry_{pq} = commonLiteral(rowHeader_p, columnHeader_q)$$

This structure can be used as the premise for the “synthesis” (introduced in [1]) and shown in Figure 5.

The fields (entries) only, part of Table IV are considered as the Akers array combination for evaluating $A_i > B_i$. The f_i in Figure 5 represents the outcome of this comparison.

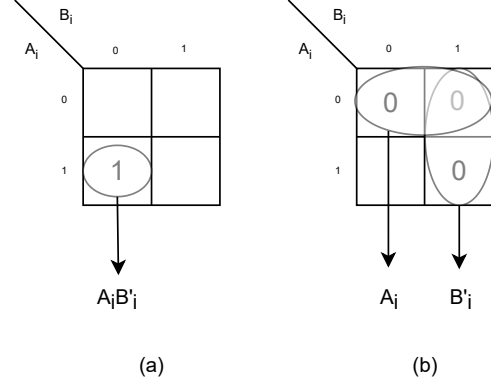


Fig. 4: K-maps for $A_i > B_i$ in (a) SOP and (b) POS forms

TABLE IV: Synthesis of $A_i > B_i$

	A_i	B'_i
AB'_i	A_i	B'_i

It can be observed that $\forall i \in \{n-1, n-2, \dots, 1, 0\}$, f_i can be computed in parallel. Now, there are two ways to proceed.

In the 1st approach, the most significant bit pairs of A and B is compared first. If the decision ($A > B$) is found, there is no need to compare the least significant bits. It removes the need for “parallel comparison” of all corresponding bit pairs.

The 2nd approach takes all the results of the parallel comparisons and uses a final linear Akers array, with space complexity $\mathcal{O}(n)$, to get the final result. This n stands for the n -bit numbers.

The branch divergence problem can be found in the 1st approach, and that can be solved using the 2nd approach [4]. Taking all these into consideration, the 2nd approach is provided for the current work. The 1-bit final comparison result $g = A_{n-1:0} > B_{n-1:0}$ can be computed as illustrated in Figure 6.

Theorem 1: The Akers array illustrated in Figure 6 implements the function $\alpha(z_0, \dots, z_{n-1}) : \{0, 1\}^n \rightarrow \{0, 1\}$, as given by:

$$\alpha = z_{n-1} \vee (z'_{n-1} \wedge z_{n-2}) \vee (z'_{n-1} \wedge z'_{n-2} \wedge z_{n-3}) \vee \dots \vee ((\bigwedge_{i=1}^{n-1} z'_i) z_0) \quad (1)$$

Proof 1: According to S. B. Akers [1], the z -input of 0 to a single Akers array cell results in row elimination, whereas 1 as the z -input results in column elimination and the outputs of each cell of such a row and each cell of such a column are 0 and 1, respectively. For the array in Figure 6, z inputs are given by $z_i = f_i$. In the topmost cell, the z input is f_{n-1} . If $f_{n-1} = 1$, using column elimination, the output $g = 1$. Otherwise, if $f_{n-1} = 0$, row elimination is performed, and so, rest of the array should be considered. This gives the 1st

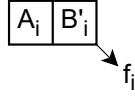


Fig. 5: Akers array for $f_i = A_i > B_i$

component of the sum in equation 1. The 2^{nd} cell from the top would be considered only if $f_{n-1} = 0$. Therefore, it gives the 2^{nd} component $f'_{n-1}f_{n-2}$. Likewise, moving downwards in the same manner, we can obtain the rest of the components.

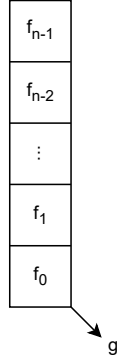


Fig. 6: Akers array for $g = A_{n-1:0} > B_{n-1:0}$

The g is utilized in an analogous style as the “select” for the “mux” (multiplexing) operations (n -bit 2-1 mux) to determine the greater and the smaller one between A and B :

$$Y_{n-1:0} = \max(A_{n-1:0}, B_{n-1:0}) \quad (2)$$

$$X_{n-1:0} = \min(A_{n-1:0}, B_{n-1:0}) \quad (3)$$

For a single bit, say i^{th} ,

$$Y_i = \max(A_i, B_i) = g'B_i + gA_i \quad (4)$$

Similarly,

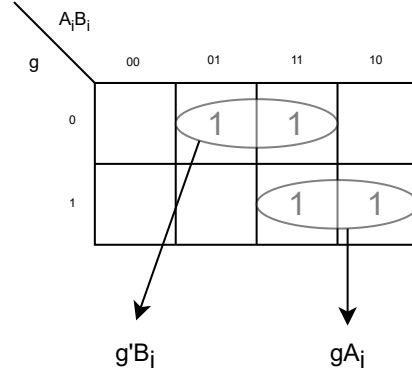
$$X_i = \min(A_i, B_i) = gB_i + g'A_i \quad (5)$$

Table V represents the truth table for Equation 5.

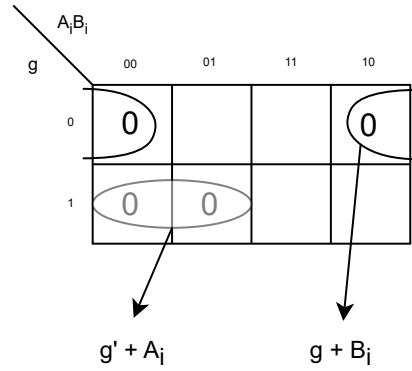
TABLE V: Truth table for $Y_i = g'B_i + gA_i$

g	A_i	B_i	Y_i
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

The K-maps for the expression $Y_i = g'B_i + gA_i$ in its SOPs and POSs forms are shown in Figure 7a and 7b, respectively. The “synthesis” of $Y_i = g'B_i + gA_i$ is shown in Table VI.



(a) SOPs



(b) POSs

Fig. 7: K-maps for $g'B_i + gA_i$ in (a) SOPs and (b) POSs

TABLE VI: Synthesis of $Y_i = g'B_i + gA_i$

	$g + B_i$	$g' + A_i$
$g'B_i$	B_i	g'
gA_i	g	A_i

It enables us to create the Akers array for $Y_i = \max(A_i, B_i)$ as depicted in Figure 8a. Similarly, we can obtain the Akers' array for $X_i = \min(A_i, B_i)$ as shown in Figure 8b can be obtained.

There can be the parallel (spatial) realization of the max/min operations for all the n -bit pairs. Now, the task is to overwrite A with X , and B with Y .

Now, various general-purpose sorting algorithms can be realized with the Akers array. implemented. For simplicity, the bubble sort is considered for the current work.

Suppose, four linearly organized 4-bit numbers —

- 1) $P = P_3P_2P_1P_0$
- 2) $Q = Q_3Q_2Q_1Q_0$

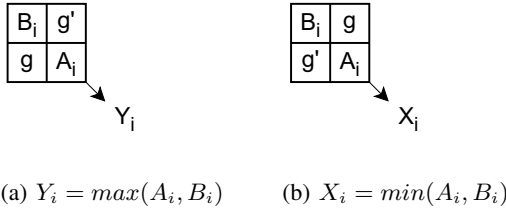


Fig. 8: Akers' array for $\min$ and $\max$

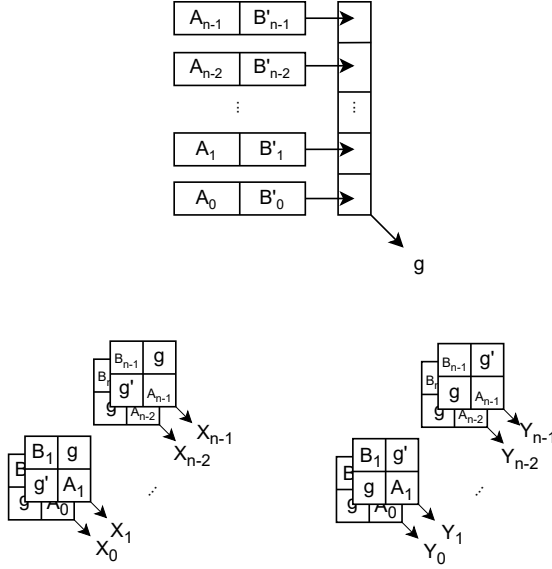


Fig. 9: Overall operations [$X = \min(A, B)$,
 $Y = \max(A, B)$]

3) $R = R_3 R_2 R_1 R_0$

4) $S = S_3 S_2 S_1 S_0$

Let consider that the unsorted list is $[P, Q, R, S]$.

First pass: the entire process (the procedure illustrated in Figure 9 and the overwrite operation) is performed on P and Q . Let this results in P' in place of P and Q' in place of Q . After that, it is applied on Q' and R which produces Q'' and R' replacing Q' and R , respectively, if applicable. Finally, it is applied on R' and S , yields R'' and S' in the given order.

Therefore, after the first pass, the sequence becomes $[P', Q'', R'', S']$. According to the bubble sort algorithm, the fourth number, i.e. S' , is in the correct position. That is why there is no need to consider the fourth number in the next pass.

Second pass: The entire process, as in first pass, is applied on P' and Q'' yielding P'' and Q''' . Next, Q''' and R'' undergo the same process and leads to Q'''' and R''' .

Consequently, after the second pass, the resulting sequence is $[P'', Q''', R''', S']$. This time, the third number, i.e., R''' is in the correct position so, it is skipped in the next pass.

Third pass: once again the process is carried out on P'' and Q'''' yielding P''' and Q''''' respectively. Eventually, the sequence after third pass becomes $[P''', Q''''', R''', S']$. In this instance, the second number, i.e., Q''''' , is in the correct position. Therefore, it can be skipped. Additionally, according to the bubble sort algorithm, a single element is always sorted. So, no extra passes will be needed for the remaining number.

The sorting concludes and the sorted sequence is $[P''', Q''''', R''', S']$.

V. IN-MEMORY SEARCHING IN AKERS ARRAY

Here, two n -bit numbers, A and B , are considered in a manner similar to that in Section IV-B. Let us further assume that A is one of the “keys” and B is the “query” of this “search” procedure. The i^{th} bits of A and B , i.e., A_i and B_i , are compared to check whether they are equal. The truth table for $A_i = B_i$ is represented as Table VII.

TABLE VII: Truth table for $A_i = B_i$

A_i	B_i	$A_i = B_i$
0	0	1
0	1	0
1	0	0
1	1	1

TABLE VIII: Synthesis of $A_i = B_i$

	$A'_i + B_i$	$A_i + B'_i$
$A'_i B'_i$	A'_i	B'_i
$A_i B_i$	B_i	A_i

As in Section IV-B, are carried out here. In this case, the obtained SOPs are — i) $A'_i B'_i$, and ii) $A_i B_i$. On the other hand, the POSs are — i) $A'_i + B_i$, and ii) $A_i + B'_i$. The synthesized Akers array for $A_i = B_i$, following Table VIII is illustrated in Figure 10.

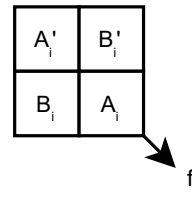


Fig. 10: Akers array for $A_i = B_i$

The process enables to design Akers logic circuit for all n pair of bits of A and B . The bit pairs are compared in parallel. These n comparison results are then chained together in Akers array as depicted in Figure 11 to obtain the final result g .

Theorem 2: The Akers array illustrated in Figure 11 implements the function $\beta(z_0, \dots, z_{n-1}) : \{0, 1\}^n \rightarrow \{0, 1\}$, as given by:

$$\begin{aligned}
\beta &= z_0 \wedge (z'_0 \vee z_1) \wedge (z'_0 \vee z'_1 \vee z_2) \wedge \\
&\quad \dots \wedge ((\bigvee_{i=0}^{n-2} z'_i) z_{n-1}) \\
&= z_0 \wedge z_1 \wedge z_2 \wedge \dots \wedge z_{n-2} \wedge z_{n-1}
\end{aligned} \tag{6}$$

The proof of this equation can be produced in a way very similar to that of the Equation 1.

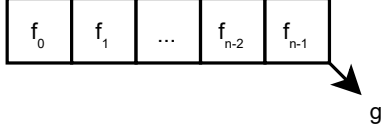


Fig. 11: Akers array for $g = (A_{n-1:0} = B_{n-1:0})$

The overall process is illustrated in Figure 12. It could be considered as a “partial” search because the query is supposed to be matched with all the keys that are present.

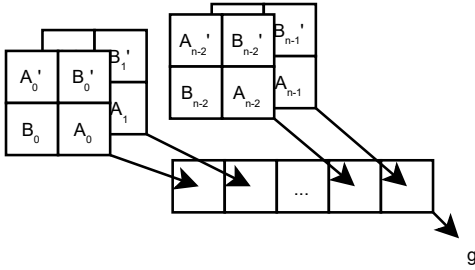


Fig. 12: Overall comparison operation for searching

Applying this process to each of the keys would vary from case to case. It could be done in a linear manner or even in parallel.

The “complete search” consists of “partial search” of the “query” with each of the keys, followed by a final Akers array operation — to combine the results obtained by all the “partial searches” and to decide if any of them is succeeded. This final step is depicted in Figure 13. The figure assumes the presence of m keys, say, $\{K_0, K_1, \dots, K_{m-2}, K_{m-1}\}$. The “partial search” with “key” K_i gives the output g_i . In this case, g_i is the z input of the i^{th} cell (0-indexed) of the Akers array depicted in Figure 13. The final result of the “complete search” operation is h .

VI. REALIZATION OF IN-MEMORY COMPUTATION

This section reposts the realization of the Akers array based sorting/searching in-memory. This realization propose partitioning of the Akers array memory structure into two sections. Each of these sections has some operations to perform.

The first section supports read/write of data. No further operation would be allowed in this partition. For reference, let us call this section **Stasis**.

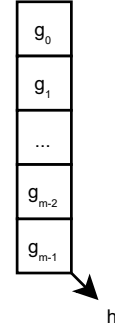


Fig. 13: The complete search operation

The second section inherits all the properties of the first section and also realizes the elementary cell operations of Akers array. Let us call this section **Flux**.

In a normal situation, the combined Stasis and Flux sections could behave as memory array. But, if required, the additional properties of the Flux section are triggered.

A simplistic illustration for such a partition is given by Figure 14. This part opens up new avenues of experimentation.

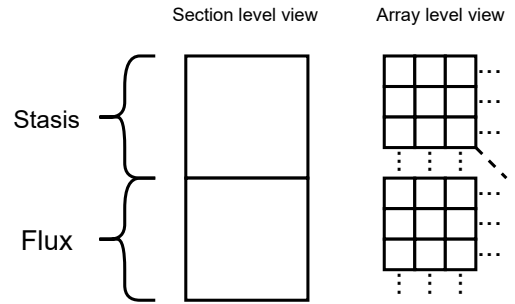


Fig. 14: Partition of memory array

The Flux section is further partitioned into a few subsections based on the particular shapes of the required Akers arrays.

Along the way through the sections IV and V, we utilized Akers arrays of the following shapes:

- 1) 1×2
- 2) 2×2
- 3) $n \times 1$
- 4) $1 \times n$

Here, n is the number of bits to represent an n -bit number. Furthermore, let us assume that the memory array is n -bit addressable. To make it simple and easy, n could be 8, turning it into a byte-addressable memory.

Such a subsection for 1×2 arrays is shown in Figure 15. Each triggered 1×2 Akers array in this subsection would be padded with a column on the right and a row at the bottom - forming a 2×3 array block. The bottom-right cell would be chosen as the target cell from where the output of the array

operation would be “read”. This kind of setup is to avoid any inter-array data corruption.

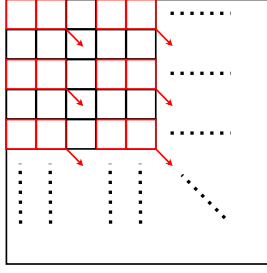


Fig. 15: The 1×2 array subsection

Similarly, a subsection of 2×2 arrays is formed in Figure 16. In this case, the blocks are 3×3 arrays.

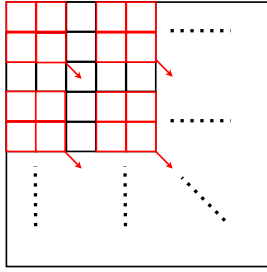


Fig. 16: The 2×2 array subsection

Figure 17 depicts a subsection for $1 \times n$ arrays. It requires a $2 \times n$ array to form a block. The $n \times 1$ array subsection is not required unless it would become strictly necessary because the $n \times 1$ array can be obtained by flipping all the external x and y inputs of the $1 \times n$ array cells.

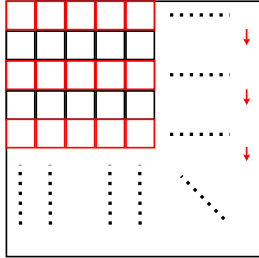


Fig. 17: The $1 \times n$ array subsection

VII. CONCLUSION

This paper demonstrates how the Akers logic array can be effectively utilized for in-memory sorting and searching, leveraging its inherent parallelism and reconfigurable structure. By extending its capabilities to support both binary and non-binary data formats, the Akers array proves to be a flexible and powerful component for data-intensive operations. The comparative analysis of sorting mechanisms and the implementation of search strategies—both linear and binary—highlight

the potential of Akers-based architectures in reducing memory bottlenecks and improving overall computational efficiency. Furthermore, the proposed in-memory model, featuring the Stasis and Flux partitioning, offers a conceptual pathway toward more dynamic and adaptable memory systems. These findings not only validate the utility of Akers arrays in current computing environments but also open up possibilities for future exploration in hardware-level optimizations for emerging data-centric applications.

REFERENCES

- [1] S. B. Akers, “A rectangular logic array,” *IEEE Transactions on Computers*, vol. C-21, no. 8, pp. 848–857, 1972.
- [2] Y. Levy, J. Bruck, Y. Cassuto, E. Friedman, A. Kolodny, E. Yaakobi, and S. Kvatinsky, “Logic operations in memory using a memristive akers array,” vol. 45, pp. 1429–1437, Nov. 2014. Publisher Copyright: © 2014 Elsevier Ltd. All rights reserved.
- [3] E. Yaakobi, A. A. Jiang, and J. Bruck, “In-memory computing of akers logic array,” in *2013 IEEE International Symposium on Information Theory*, pp. 2369–2373, 2013.
- [4] T. D. Han and T. S. Abdelrahman, “Reducing branch divergence in gpu programs,” in *Proceedings of the Fourth Workshop on General Purpose Processing on Graphics Processing Units, GPGPU-4*, (New York, NY, USA), Association for Computing Machinery, 2011.

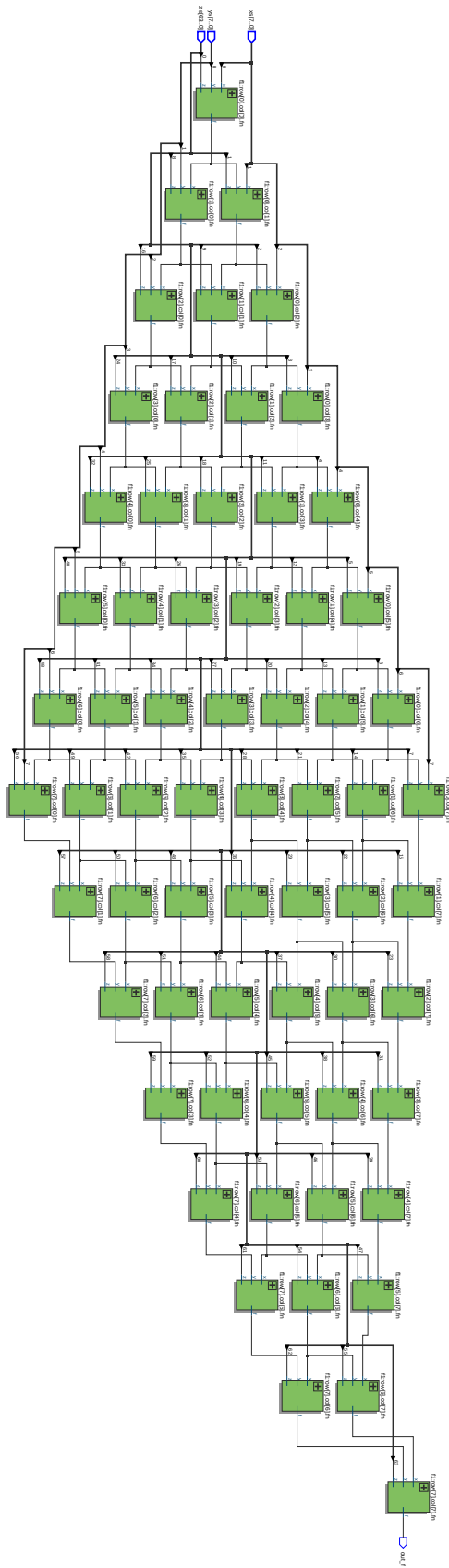


Fig. 18: 8×8 Akers array RTL design